

회귀 & 분류

Kuggle 13기 안시현 | 엄창현

OVERVIEW

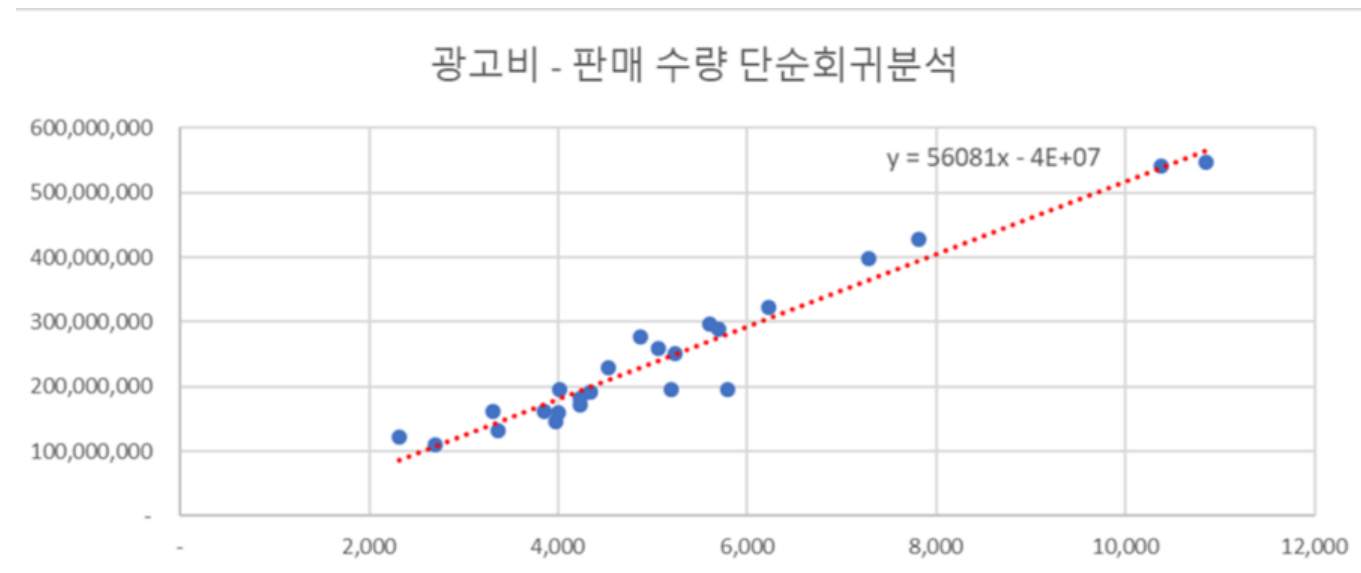
프레젠테이션 구성

- 1 회귀분석 정의 및 종류
- 2 회귀 평가 지표
- 3 과대적합 & 과소적합
- 4 파이썬을 활용한 회귀
- 5 분류의 정의 및 종류
- 6 분류 알고리즘
- 7 분류 평가 지표

회귀분석의 정의

회귀 분석이란?

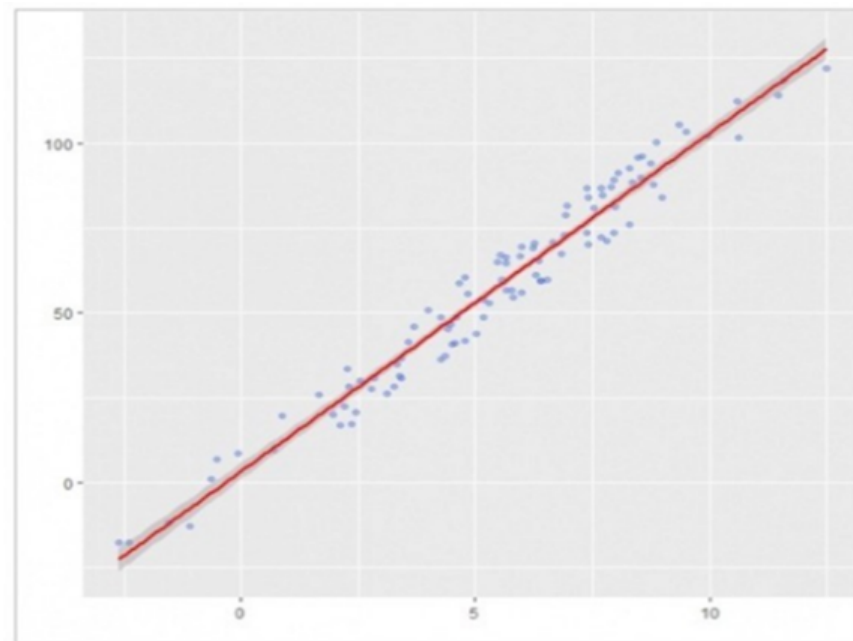
하나 또는 그 이상의 독립변수와 종속변수 사이의 인과관계를 수학적 모형으로 설명하고, 이를 통해 새로운 데이터를 예측하는 통계 기법



판매수량: 종속변수(반응변수), 광고비: 독립변수(설명변수)
→ 목적: 변수 간의 관계(경향성) 파악 및 미래 값 예측

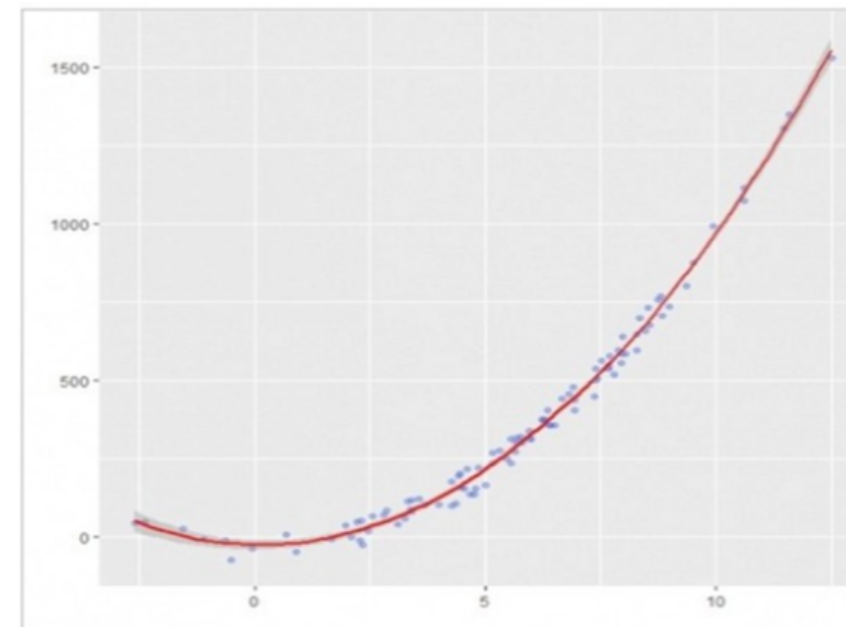
회귀분석의 분류

독립변수 개수 → 단순회귀 VS 다중회귀



단순회귀식

$$Y = \beta_0 + \beta_1 X + \epsilon$$



다중회귀식

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \epsilon$$

회귀 평가 지표

01. MSE (Mean Squared Error, 평균 제곱 오차)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

02. RMSE (Root Mean Squared Error, 평균 제곱근 오차)

$$\text{RMSE} = \sqrt{\text{MSE}}$$

03. MAE (Mean Absolute Error, 평균 절대 오차)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

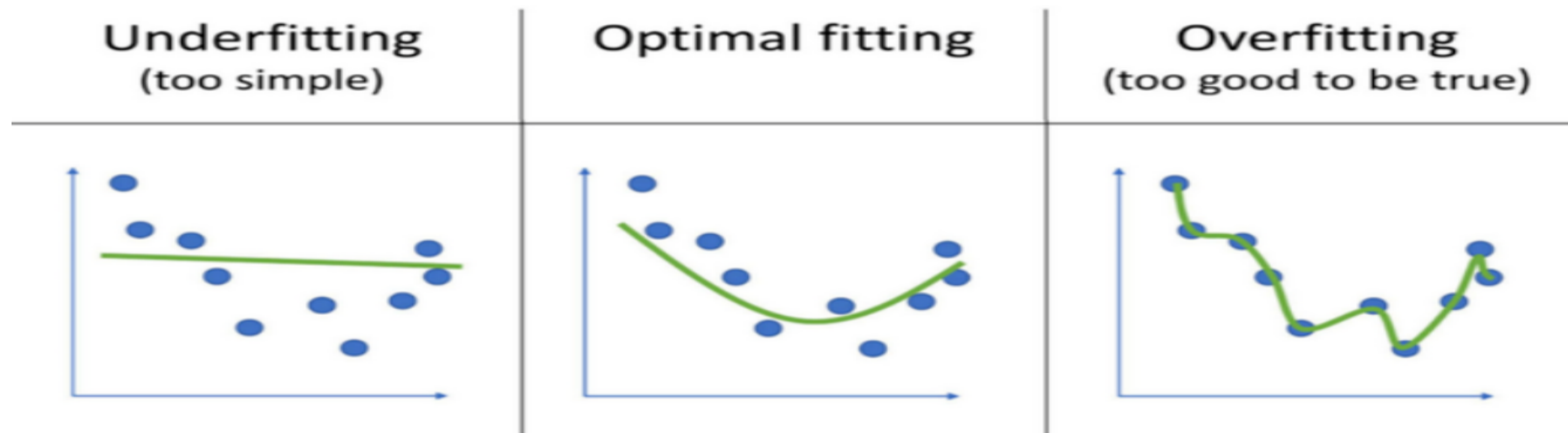
04. R² (결정계수, 설명력)

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

과대적합 & 과소적합

1. 편향과 분산(Bias-Variance Tradeoff)

- 편향(Bias): 예측값(선)과 실제값(점) 간의 차이(정확성)
- 분산(Variance): 동일한 모델에 다른 데이터셋 사용 시 예측값(선)의 변동 정도(일관성)



편향이 크면 모델이 데이터의 패턴을 충분히 학습하지 못함

편향과 분산 적절함

분산이 크면, 모델이 훈련 데이터에 지나치게 맞춰짐

과대적합 & 과소적합

2. 훈련 세트와 테스트 세트

- 1) 훈련 세트를 활용하여 모델 훈련 후, 훈련 세트와 테스트 세트에서 평가
- 2) 훈련 세트와 테스트 세트를 보통 7:3, 8:2로 분류



- 3) 훈련 세트로 훈련했기 때문에 훈련 세트의 점수가 더 높게 나오는 경향이 있음
(score: 결정계수 값, 1에 가까울수록 좋음)
- 4) 훈련 세트점수가 테스트 세트 점수보다 훨씬 높은 경우 → 과대적합
- 5) 테스트 세트 점수가 훈련 세트 점수보다 높거나, 두 점수 모두 낮은 경우 → 과소적합

과대적합 & 과소적합

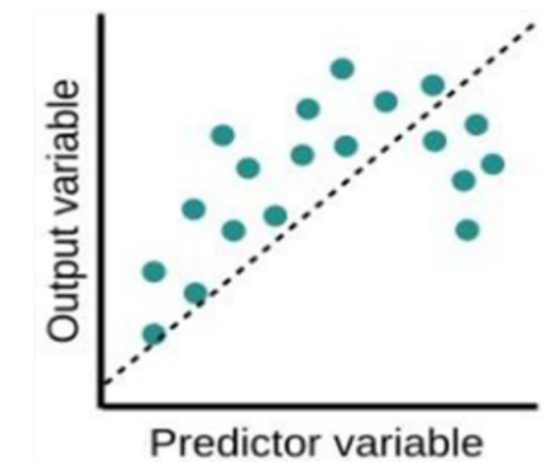
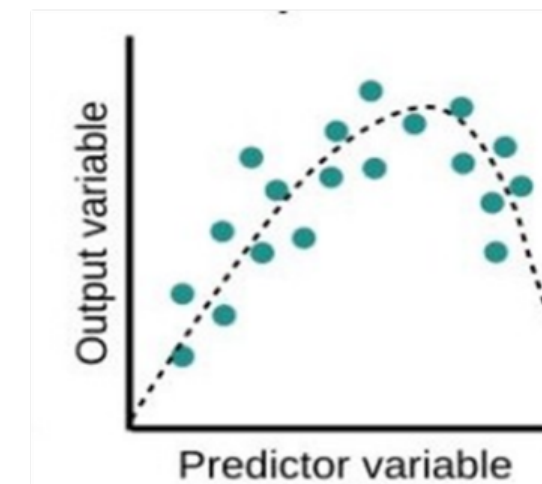
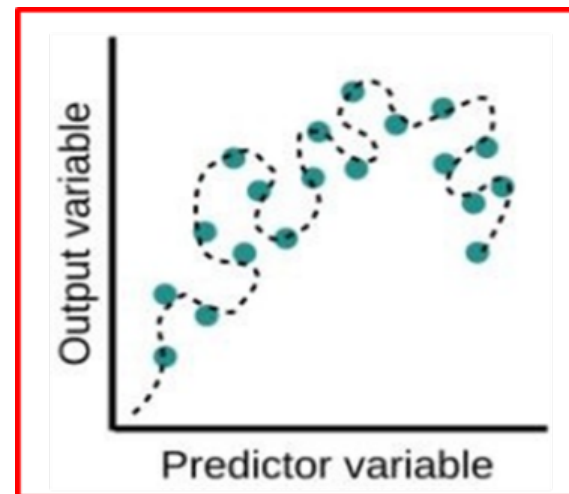
3. 과대적합(Overfitting)

- 정의

: 모델이 학습 데이터에 '과하게' 맞추어져 실전 예측력이 떨어지는 상태(노이즈까지 학습)

- 특징

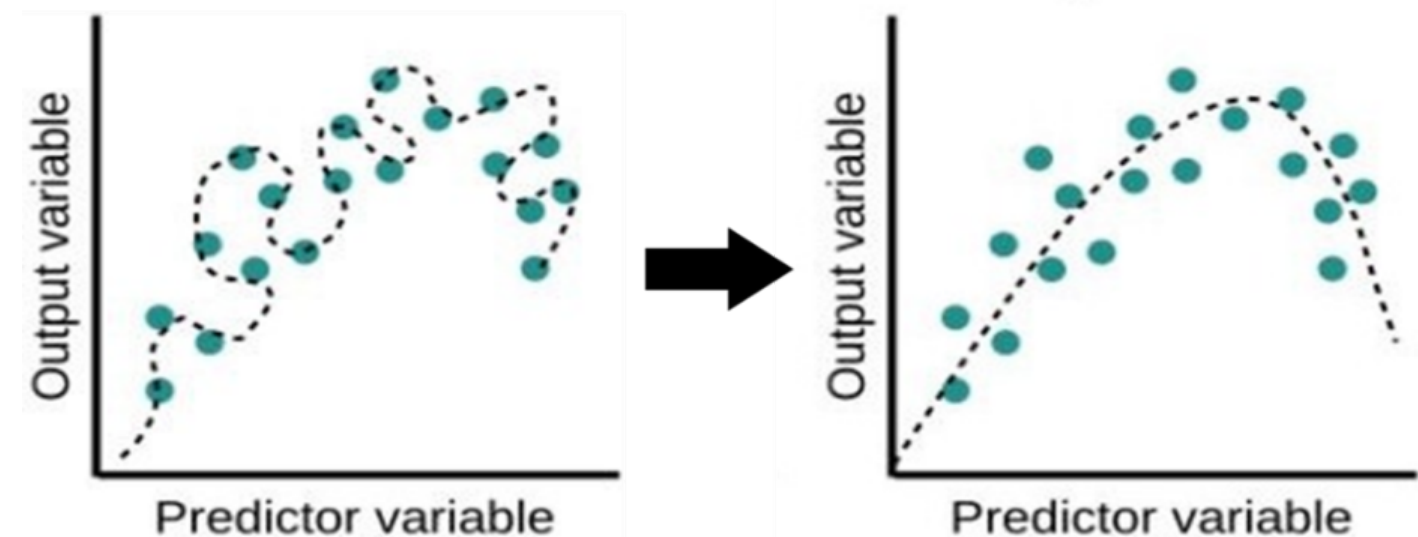
- 1) 훈련 세트에 대한 성능 >> 테스트 데이터에 대한 성능
- 2) 설명 변수가 너무 많음
- 3) 복잡한 모델에서 자주 발생 (ex. 너무 많은 변수, 너무 깊은 신경망 등)



과대적합 & 과소적합

4. 과대적합(Overfitting) 해결 방법

- 1) 모델 복잡도 감소 → 모델 단순화
- 2) 더 많은 데이터 사용
- 3) Early Stopping 적용 → 과대적합 전에 학습 중단
- 4) 정규화 적용 → L1, L2 정규화 활용



과대적합 & 과소적합

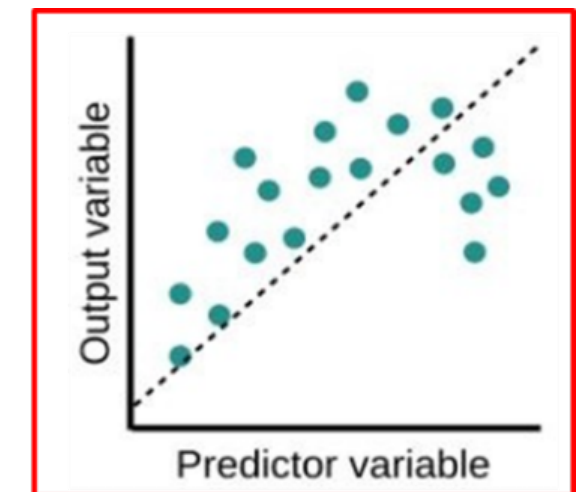
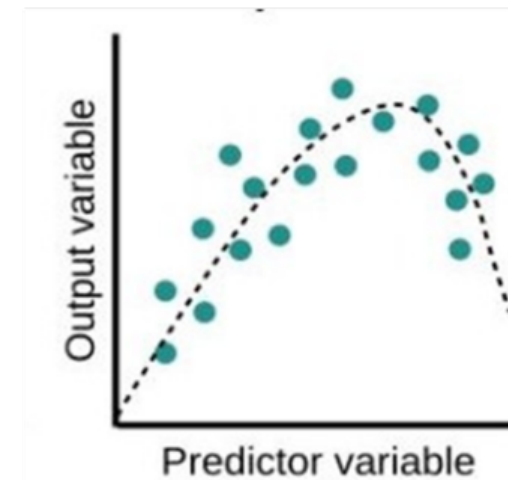
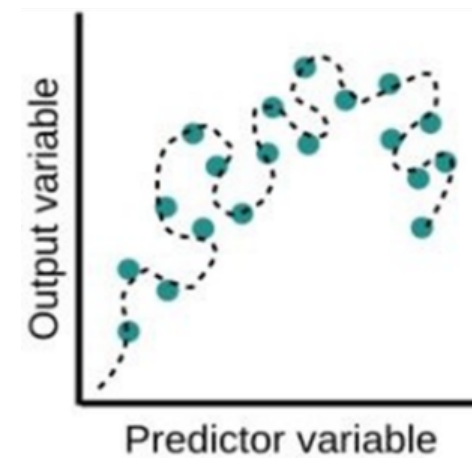
5. 과소적합(Underfitting)

- 정의

:모델이 너무 단순하여 데이터에 내재된 기본적인 규칙이나 패턴조차 학습하지 못한 상태

-특징

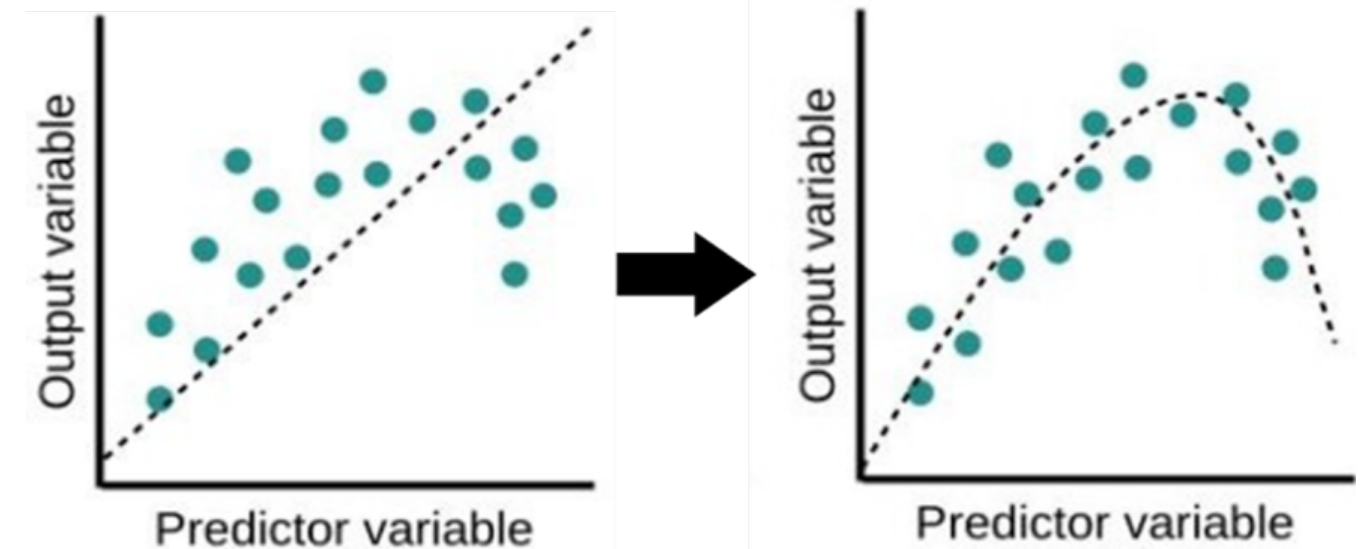
- 1) 훈련 세트에 대한 성능 << 테스트 세트에 대한 성능
- 2) 훈련 세트에 대한 성능 ↓ 테스트 세트에 대한 성능 ↓
- 3) 너무 단순한 모델에서 자주 발생 (ex. 선형 회귀로 비선형 관계를 학습하려는 경우)



과대적합 & 과소적합

6. 과소적합(Underfitting) 해결 방법

- (1) 모델 복잡도 증가 → 더 복잡한 모델 사용
- (2) 더 많은 Feature 추가 → 유의미한 변수 추가



파이썬을 활용한 회귀

< LinearRegression을 이용한 예측 코드 >

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 데이터 준비
X =
y =

# 데이터 분할
# Test size는 0.3 (난수는 자유)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

파이썬을 활용한 회귀

< LinearRegression을 이용한 예측 코드 >

모델 생성

```
lr = LinearRegression()
```

학습

```
lr.fit(X_train, y_train)
```

예측

```
y_preds = lr.predict(X_test)
```

평가

```
mse = mean_squared_error(y_test, y_preds)
```

```
rmse = np.sqrt(mse)
```


파이썬을 활용한 회귀

< LinearRegression을 이용한 예측 코드 >

MSE값 출력

```
print('MSE: {0:.3f}, RMSE:{1:3f}'.format(mse, rmse))
```

```
print('Variance score:
```

```
{0:.3f}'.format(r2_score(y_test, y_preds)))
```

절편 값

```
print('절편 값:', lr.intercept_)
```

회귀계수 값

```
print('회귀계수 값:', np.round(lr.coef_, 1))
```

파이썬을 활용한 회귀

< 과적합 방지를 위한 릿지 모델 >

```
from sklearn.linear_model import Ridge

ridge = Ridge() # 릿지 모델 불러오기
ridge.fit(train_scaled, train_target) # 모델 훈련하기
print(ridge.score(train_scaled, train_target)) # 훈련세트 점수
print(ridge.score(train_scaled, test_target)) # 테스트세트 점수

# Alpha 값으로 규제의 강도 조절 가능
ridge = Ridge(alpha=10)
```

파이썬을 활용한 회귀

< 과적합 방지를 위한 라쏘 모델 >

```
from sklearn.linear_model import Lasso
Lasso = Lasso()
lasso.fit(train_scaled, train_target) # 모델 훈련하기
print(lasso.score(train_scaled, train_target)) # 훈련세트 점수
print(lasso.score(train_scaled, test_target)) # 테스트세트 점수

# Alpha 값으로 규제의 강도 조절 가능
lasso = Lasso(alpha=10)
```




분류의 정의

분류란?

주어진 데이터의 특성을 파악하여 데이터가 어떤 카테고리에 속하는지 예측하는 과정으로, 새로운 데이터가 들어왔을 때, 이 데이터가 어느 그룹에 속할 지 정확히 예측하는 것이 목표이다.

분류 VS 회귀

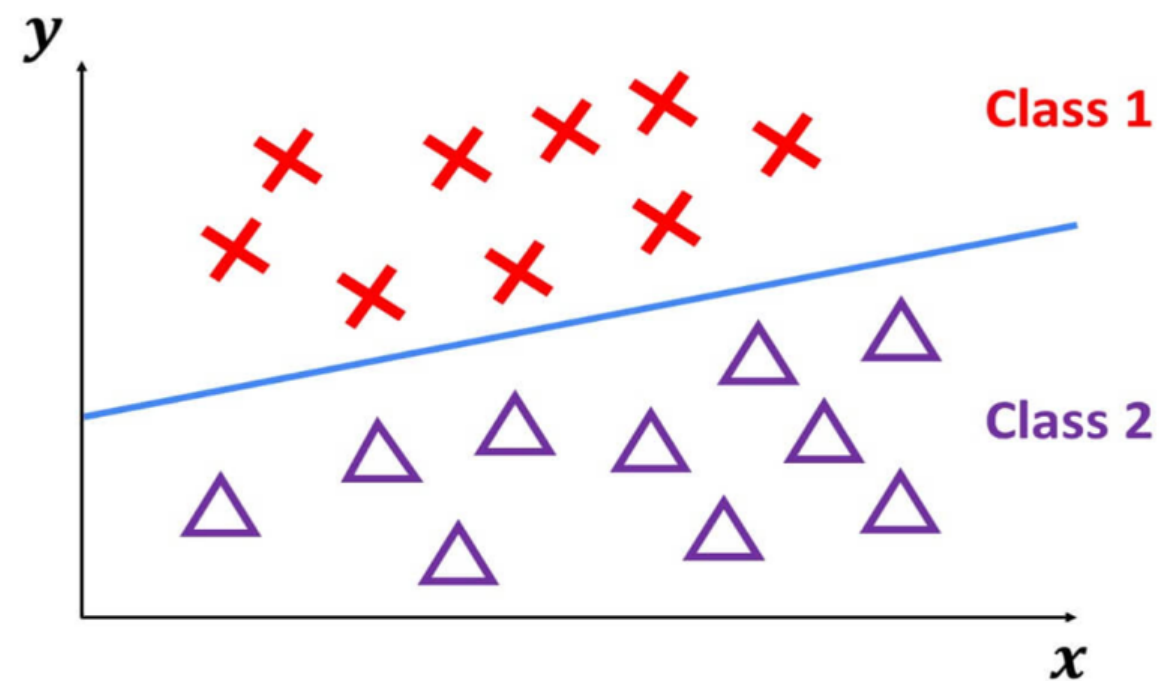
모델 유형 구분		출력값	
분류 모델	내일 날씨가 추울까요? 더울까요?	덥다 (혹은) 춥다	→ 문자 형태 (비연속성)
회귀 모델	내일 기온은 몇 도일까요?	32도	→ 숫자 형태 (연속성)

분류의 종류

- 카테고리의 개수 → 이진 분류 VS 다중 분류

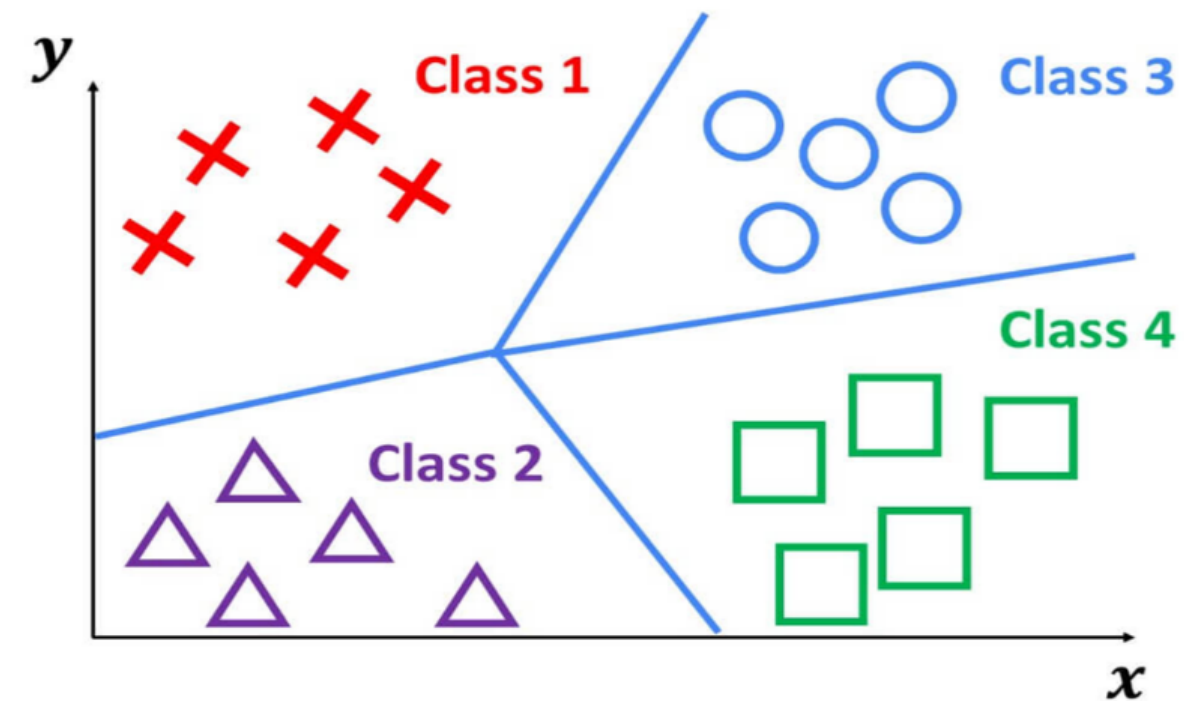
이진 분류 : 분류 카테고리가 2개인 경우
EX) 질병 양성인지 음성인지 예측

Binary Classification



다중 분류 : 분류 카테고리가 여러가지인 경우
EX) 꽃의 품종이 무엇인지 예측

Multiclass Classification



분류 알고리즘

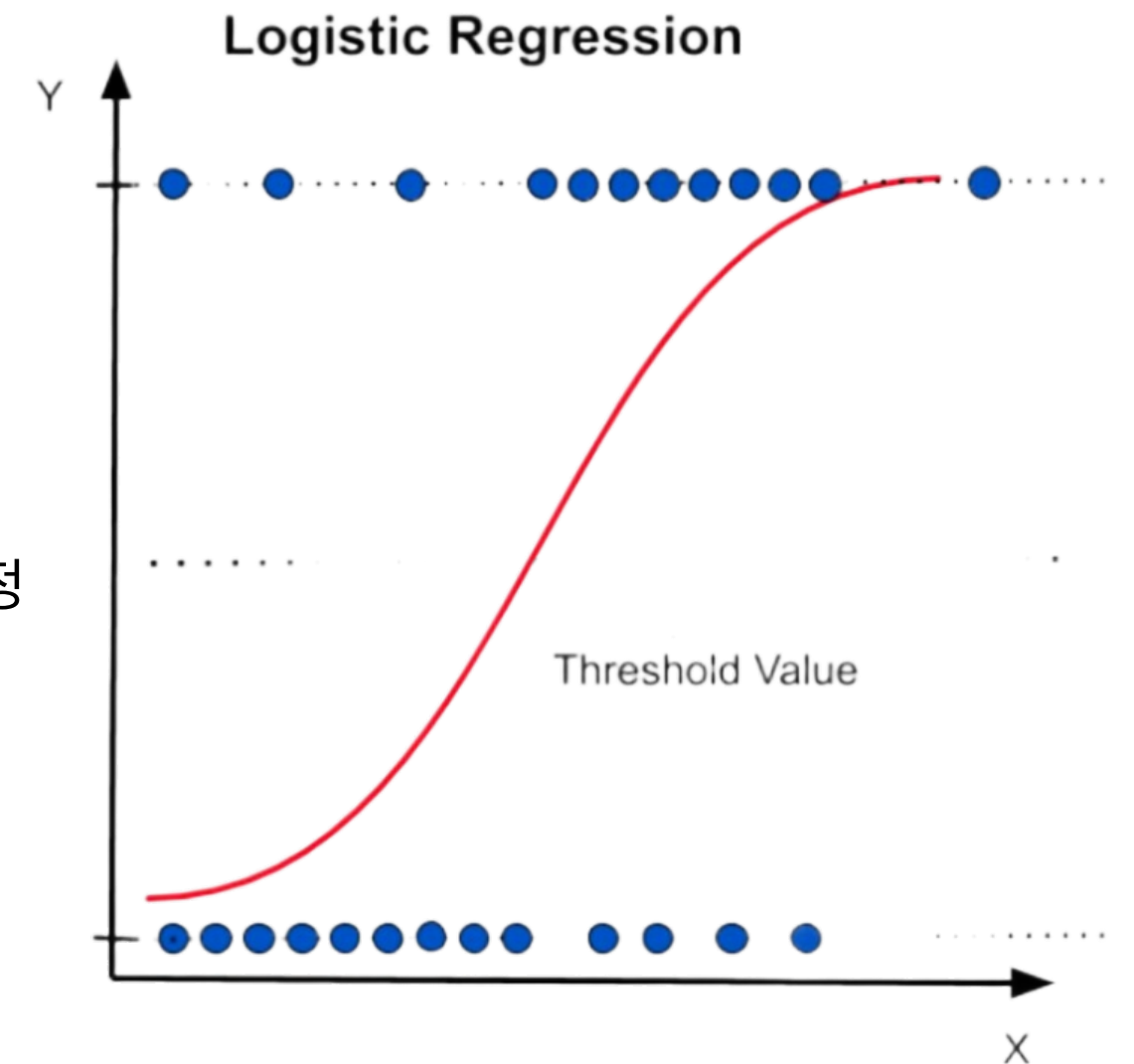
1. 로지스틱 회귀 (LOGISTIC REGRESSION)

이진 분류 문제를 해결하는데 적합한 선형 모델 기반의 분류 알고리즘

시그모이드(SIGMOID) 함수를 사용해 결과값을 확률로 변환 후 임계값 기준으로 클래스 결정

장점 : 모델이 단순하고 속도가 빠르고 비교적 결과 해석이 쉬움

단점 : 비선형적이거나 복잡한 데이터를 분류하는 데는 한계가 존재



분류 알고리즘

2. K-최근접 이웃 (K-NEAREST NEIGHBOR, KNN)

가까운 이웃을 보고 판단하는 직관적인 모델

새로운 데이터 주변에 가장 가까운 K개의 이웃을 탐색하고, 자신의 클래스를 결정

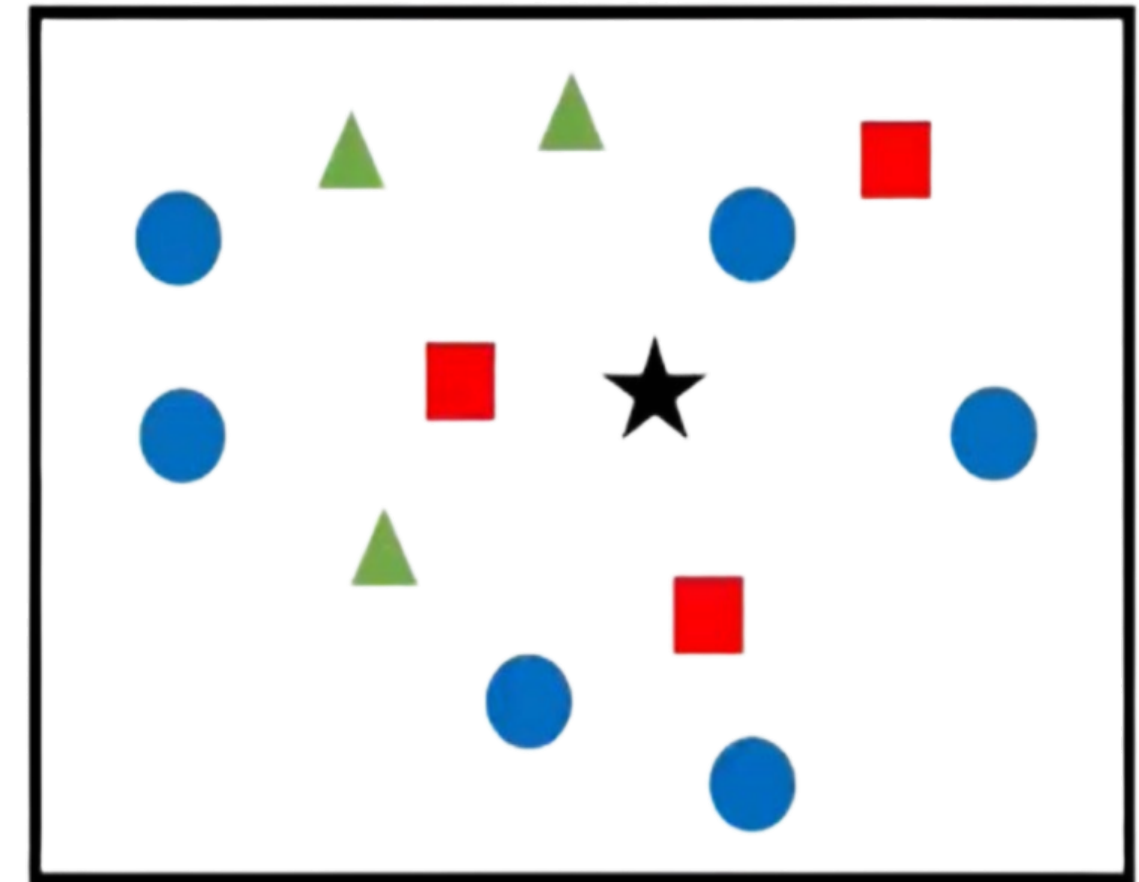
거리를 어떤 방식으로 정의할 것인가, 탐색할 이웃의 수를 몇 개로 할 것인가 논의 필요

장점 : 알고리즘이 간단하고 이해하기 쉬우며, 특별한 학습 과정이 필요 없음

단점 : 데이터가 많아질수록 계산할 거리의 양이 많아져 속도가 느려짐

K-Nearest Neighbor

- K가 3일 때 '빨간색: 2개 / 파란색: 1개'입니다.



분류 알고리즘

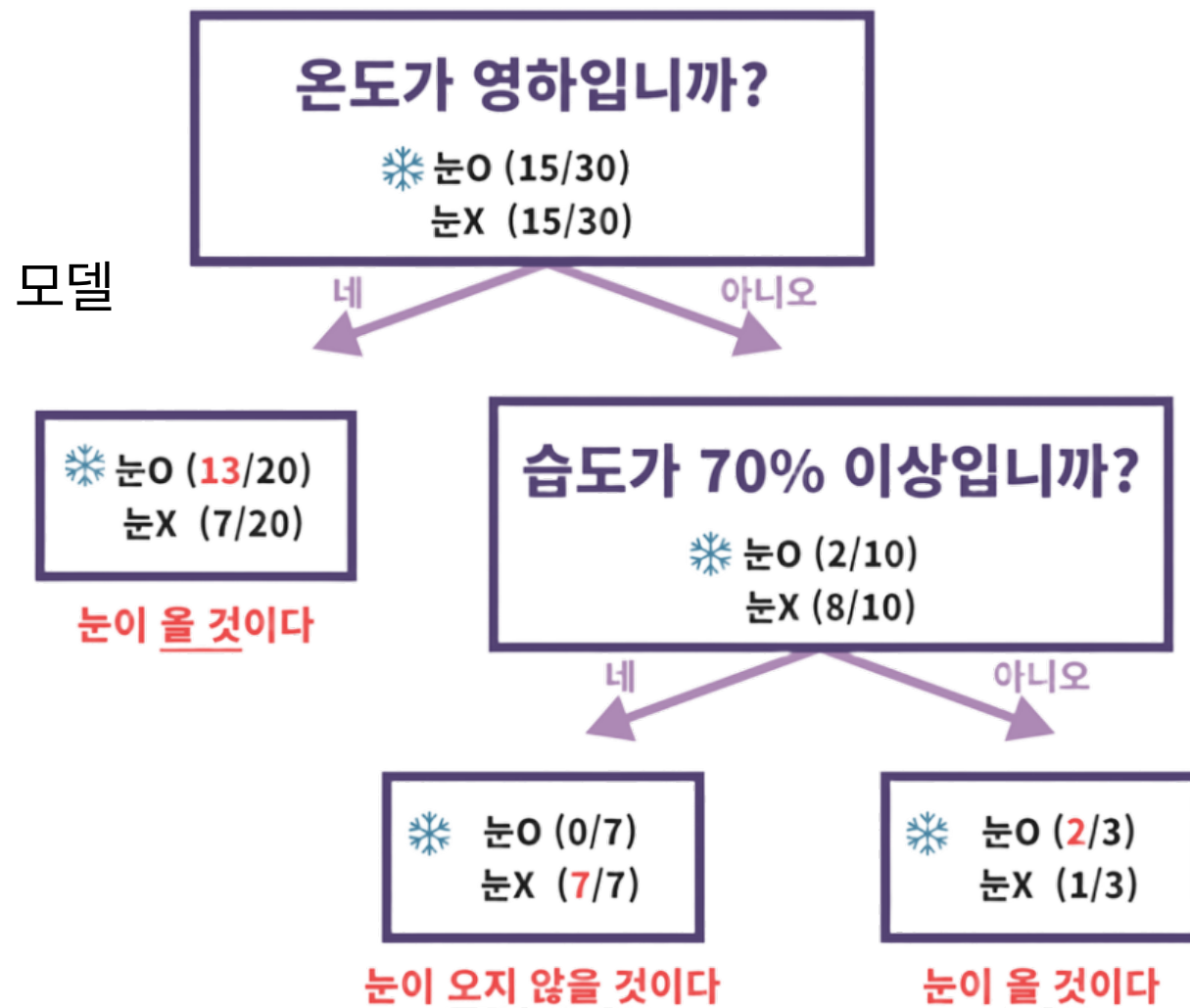
3. 의사결정나무 (DECISION TREE)

데이터를 잘 구분할 수 있는 질문을 기준으로 계속해서 하위 그룹으로 나누어 가는 모델

나무 구조로 데이터 분할 과정을 표현

장점 : 모델의 분류 과정을 시각적으로 쉽게 이해하고 해석 가능

단점 : 나무가 너무 복잡하고 깊어지면 과적합 현상이 발생하기 쉬움



분류 알고리즘

4. 랜덤포레스트 (RANDOM FOREST)

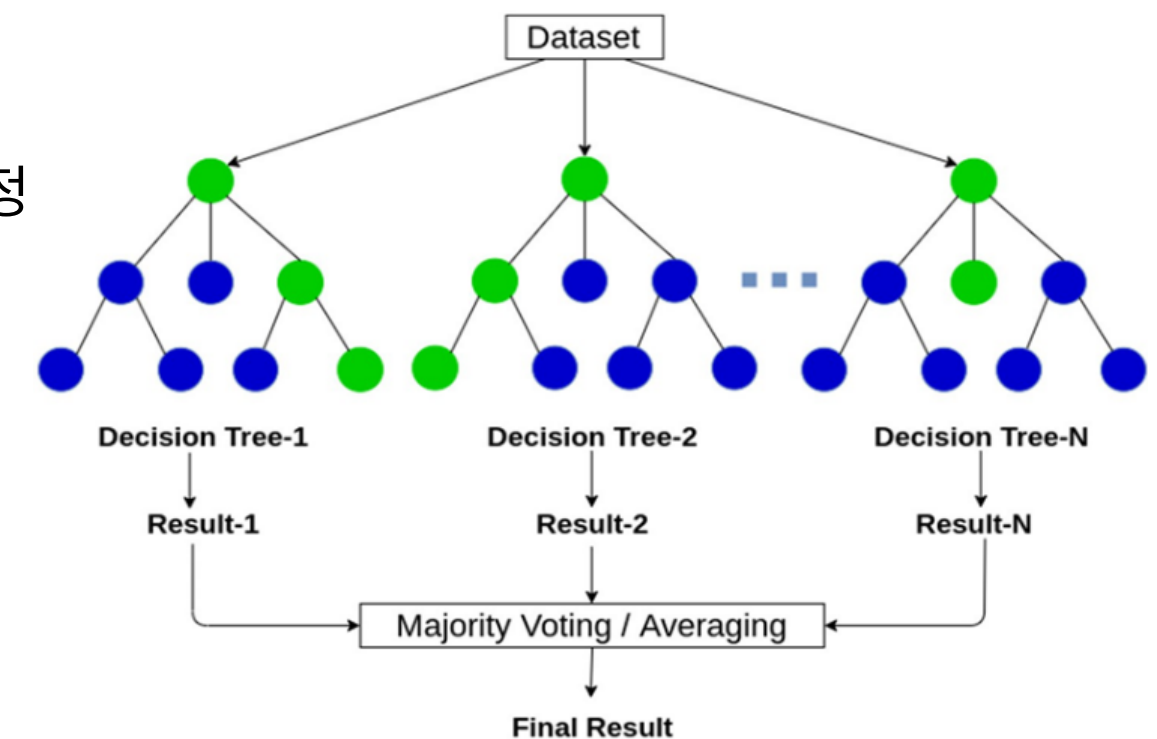
베깅(BAGGING) : 앙상블 학습의 기법 중 하나로 데이터를 복원 추출해 여러 개의 데이터 셋을 만들고, 각각의 데이터 셋으로 모델을 학습시켜 결과를 종합하는 방식

여러 개의 의사결정나무를 무작위로 생성한 후, 각 나무가 예측한 결과를 종합해 클래스 결정

장점 : 단일 의사결정나무보다 훨씬 높은 정확도를 보이고, 과적합에 강함

단점 : 여러 개의 나무를 만들기 때문에 학습 속도가 느리고 메모리 사용량이 많음

Random Forest



분류 평가 지표

- 오차 행렬

		예측 클래스	
		Positive(1)	Negative(0)
실제 클래스	Positive(1)	TP	FN
	Negative(0)	FP	TN

TP : 모델이 POSITIVE라고 예측 + 실제 정답이 POSITIVE

FP : 모델이 POSITIVE라고 예측 + 실제 정답이 NEGATIVE

TN : 모델이 NEGATIVE라고 예측 + 실제 정답이 NEGATIVE

FN : 모델이 NEGATIVE라고 예측 + 실제 정답이 POSITIVE

분류 평가 지표

1. 정확도 (ACCURACY)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

전체 데이터 중에서 모델이 올바르게 예측한 비율

데이터가 불균형한 경우 신뢰하기 어렵다는 단점 존재

2. 정밀도 (PRECISION)

$$Precision = \frac{TP}{TP + FP}$$

모델의 POSITIVE 예측이 얼마나 정밀한지 나타내는 지표

FP가 결과에 큰 영향을 미치는 모델의 경우 중요

EX) 스팸 메일 분류 모델

분류 평가 지표

3. 재현율 (RECALL)

$$Recall = \frac{TP}{TP + FN}$$

모델이 POSITIVE인 데이터를 얼마나 잘 재현했는지 확인하는 지표

FN이 결과에 큰 영향을 미치는 모델의 경우 중요

EX) 암 진단 분류 모델

4. F1 SCORE

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

RECALL과 PRECISION의 조화평균으로 둘의 균형점을 알려주는 지표

데이터가 불균형 할 때 정확도를 보완하기 좋은 지표

THANK YOU.